



DATAMITES

Global Training & Certification Institute for Data Science

DATA SCIENCE CAPSTONE PROJECT

PRCP-1001: Rice Leaf Disease Detection Using Convolutional Neural Networks (CNN)

Rubiex Internship Project

Submitted in Partial Fulfilment of the DataMites Data Science Internship Program

Project Title: Rice Leaf Disease Detection Using Convolutional Neural Networks (CNN)

Team Code: PTID-AIE-JUL-26-11142

Project Code: PRCP-1001

Project Type: Data Science Capstone Project / Rubiex Internship Project

Domain: Deep Learning — Image Classification (Computer Vision)

Dataset Source: DataMites Capstone Project Dataset

Dataset Name: Rice Leaf Disease Dataset (119 Images, 3 Classes)

Organization: DataMites Solutions Pvt. Ltd.

Submitted By: Sameeksha

Report Date: July 2026

About the Author

This capstone project has been prepared and submitted as part of the DataMites Data Science Internship Program, carried out under the Rubiex Internship Project initiative. The following section provides a brief profile of the project author.

Name	Sameeksha
Role	Data Science Intern / Capstone Project Author
Program	DataMites Data Science Internship Program
Internship Track	Rubiex Internship Project
Team Code	PTID-AIE-JUL-26-11142
Project Code	PRCP-1001
Project Title	Rice Leaf Disease Detection Using Convolutional Neural Networks (CNN)
Tools & Technologies	Python, TensorFlow / Keras, OpenCV, PIL, NumPy, Pandas, Matplotlib, Seaborn, Google Colab
Core Skills Applied	Image Preprocessing, Data Augmentation, CNN Model Building, Transfer Learning (VGG16, MobileNetV2), Model Evaluation & Comparison
GitHub Repository	github.com/Sameekshagithub/PRCP-1001-Rice-Leaf-Disease-Detection-Using-Convolutional-Neural-Networks-CNN-

As part of this internship, the author independently carried out the complete deep learning project lifecycle for the assigned capstone problem — from problem understanding and dataset exploration, through image preprocessing and augmentation, deep learning model building using CNN and transfer learning architectures, to model comparison, selection, and final prediction testing. The project reflects hands-on application of computer vision and deep learning techniques on a real-world agricultural image dataset, in line with the learning objectives of the DataMites Data Science Internship Program under the Rubiex Internship Project track.

Table of Contents

- 1. Problem Statement & Project Objective.....
- 2. Import Python Libraries.....
- 3. Download / Upload and Extract the Dataset.....
- 4. Organizing the Dataset for the CNN Model.....
- 5. Exploratory Data Analysis (EDA) — Task 1.....
- 6. Train / Validation / Test Split.....
- 7. Data Preprocessing & Data Augmentation — Task 3.....
- 8. Model Building & Training — Task 2.....
- 9. Model Comparison Report.....
- 10. Testing the Best Model on Sample Images.....
- 11. Report on Challenges Faced.....
- 12. Conclusion.....
- 13. Project Links.....

Executive Summary

Rice is one of the most widely cultivated staple food crops in the world, and its yield is significantly affected by leaf diseases. Early and accurate disease identification helps farmers apply the appropriate treatment at the right time, reducing crop loss and improving productivity. However, manual disease detection is time-consuming, subjective, and requires expert knowledge.

This capstone project, undertaken as part of the DataMites Data Science Internship Program under the Rubiex Internship Project track, develops an automated Convolutional Neural Network (CNN)-based image classification system to identify rice leaf diseases from leaf images. The dataset consists of 119 rice leaf images distributed across three disease classes — Bacterial Leaf Blight, Brown Spot, and Leaf Smut.

A complete deep learning pipeline was implemented: dataset extraction and organisation, exploratory data analysis, a 70:15:15 train/validation/test split, image preprocessing and augmentation, and the training of three models — a Baseline CNN trained from scratch, and two transfer-learning models, VGG16 and MobileNetV2, both pretrained on ImageNet. All three models were evaluated on the same held-out test set using accuracy, precision, recall, F1-score, trainable parameters, and training time.

MobileNetV2 (Transfer Learning) was selected as the best-performing and recommended production model, achieving 94.74% test accuracy with only 164,355 trainable parameters and a training time of 81.1 seconds — decisively outperforming both the Baseline CNN (31.58% accuracy) and VGG16 (36.84% accuracy). This report documents the complete methodology, results, and recommendations arising from the project.

1. Problem Statement & Project Objective

1.1 Problem Statement

Rice is one of the most widely cultivated staple food crops in the world, and its yield is significantly affected by leaf diseases. Early and accurate disease identification helps farmers apply the appropriate treatment at the right time, reducing crop loss and improving productivity. However, manual disease detection is time-consuming, subjective, and requires expert knowledge. This project develops an automated Convolutional Neural Network (CNN)-based image classification system to identify rice leaf diseases from leaf images.

1.2 Project Objectives

1. Perform Data Analysis — Analyze the dataset to understand its structure, class distribution, and image characteristics.
2. Develop a Classification Model — Build a CNN model to classify rice leaf images into three disease categories: Bacterial Leaf Blight, Brown Spot, and Leaf Smut.
3. Apply Data Augmentation — Use image augmentation techniques to improve model generalization and reduce overfitting.
4. Compare Multiple Models — Evaluate the performance of different models and identify the most suitable model for deployment.
5. Document Challenges and Solutions — Discuss the challenges encountered during model development and the techniques used to overcome them.

1.3 About the Dataset

The dataset consists of 119 rice leaf images, distributed across three disease classes (approximately 40 images per class):

Disease Class	Visual Characteristics
Bacterial Leaf Blight	Long yellow or white streaks along the leaf veins
Brown Spot	Small circular or oval brown lesions on the leaf surface
Leaf Smut	Tiny black spots scattered across the leaf

Since these diseases have distinct visual characteristics such as shape, color, and texture, a Convolutional Neural Network (CNN) is well suited for accurately classifying them from leaf images.

2. Import Python Libraries

The first step in the project is to import the required Python libraries. These libraries provide the functionality needed for data handling, image preprocessing, visualization, deep learning, and model evaluation.

Library / Module	Purpose in the Project
os, shutil, zipfile	File and folder management, ZIP extraction
NumPy, Pandas	Numerical computations and data manipulation
OpenCV (cv2), PIL	Image processing and image dimension checks
Matplotlib, Seaborn	Visualizing images and model performance
TensorFlow / Keras	Building and training the CNN, VGG16 and MobileNetV2 models
ImageDataGenerator	Image preprocessing and data augmentation
EarlyStopping, ModelCheckpoint	Optimizing the training process
classification_report, confusion_matrix	Evaluating model performance

A fixed SEED = 42 is used throughout the notebook to ensure consistent and reproducible results across all experiments.

3. Download / Upload and Extract the Dataset

The dataset was provided as a .zip file with an additional layer of compressed folders created during the Google Drive export. Therefore, extraction was performed in two stages:

6. Extract the main PRCP-1001-RiceLeaf.zip file to access the Data folder.
7. Extract the individual class-wise .zip files to obtain the actual rice leaf disease images.

This two-stage extraction ensured that the dataset was organized correctly and ready for preprocessing and model training. The outer ZIP extraction revealed a Data folder containing three class-wise ZIP files: Brown spot-20200814T055208Z-001.zip, Bacterial leaf blight-20200814T055237Z-001.zip, and Leaf smut-20200814T055530Z-001.zip, each of which was subsequently extracted to reveal the individual class-wise image folders.

3.1 Dataset Link

d3ilbtxij3aepc.cloudfront.net/projects/CDS-Capstone-Projects/PRCP-1001-RiceLeaf.zip

4. Organizing the Dataset for the CNN Model

The extracted folders contained long, inconsistent names due to the Google Drive export (e.g. "Bacterial leaf blight-20200814T055237Z-001"). To create a clean dataset structure, all images were copied into a new RiceLeaf folder with separate, clearly named sub-folders for each disease class. This organized folder structure enables Keras's `flow_from_directory` to automatically identify class labels during model training.

After reorganization, the following disease classes were confirmed present in the clean dataset folder: Brown Spot, Leaf Smut, and Bacterial Leaf Blight.

5. Exploratory Data Analysis (EDA) — Task 1

This section answers Task 1 of the problem statement: a complete data analysis of the given data. Four aspects were examined that matter most before training any image classification model: class distribution, image dimensions, visual inspection of the disease classes, and overall data quality (corrupt/unreadable files).

5.1 Class Distribution

Class	Number of Images
Brown Spot	40
Leaf Smut	39
Bacterial Leaf Blight	40

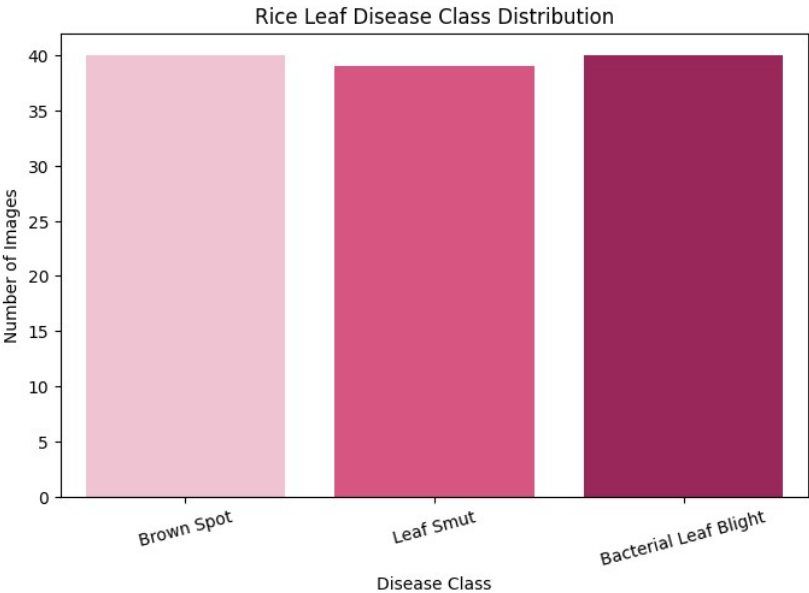


Figure 5.1 — Rice Leaf Disease Class Distribution

Total images: 119. The dataset is very close to perfectly balanced (39–40 images per class), so no class-imbalance correction (e.g. class weights or oversampling) was required.

5.2 Image Dimension Check

A sample image from each class showed markedly different dimensions:

Class	Sample Size	Mode
Brown Spot	765 × 224	RGB
Leaf Smut	3081 × 897	RGB
Bacterial Leaf Blight	3081 × 897	RGB

Checking dimensions across all 119 images confirmed substantial variation in image size, with 34 unique resolutions found in the dataset:

Statistic	Width (px)	Height (px)
Mean	2383.6	707.7
Std. Dev.	1123.5	311.7
Min	250	71
Median	3081	897
Max	3081	900

Given this wide variation in native resolutions, all images were resized to a uniform 224×224 pixels before training, ensuring compatibility with the CNN, VGG16, and MobileNetV2 architectures.

5.3 Visual Inspection

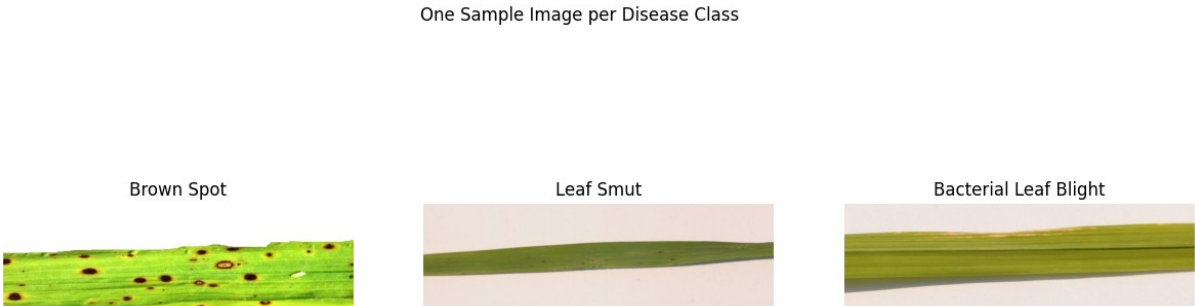


Figure 5.2 — One Sample Image per Disease Class

Within-Class Visual Variation (first 5 images per class)

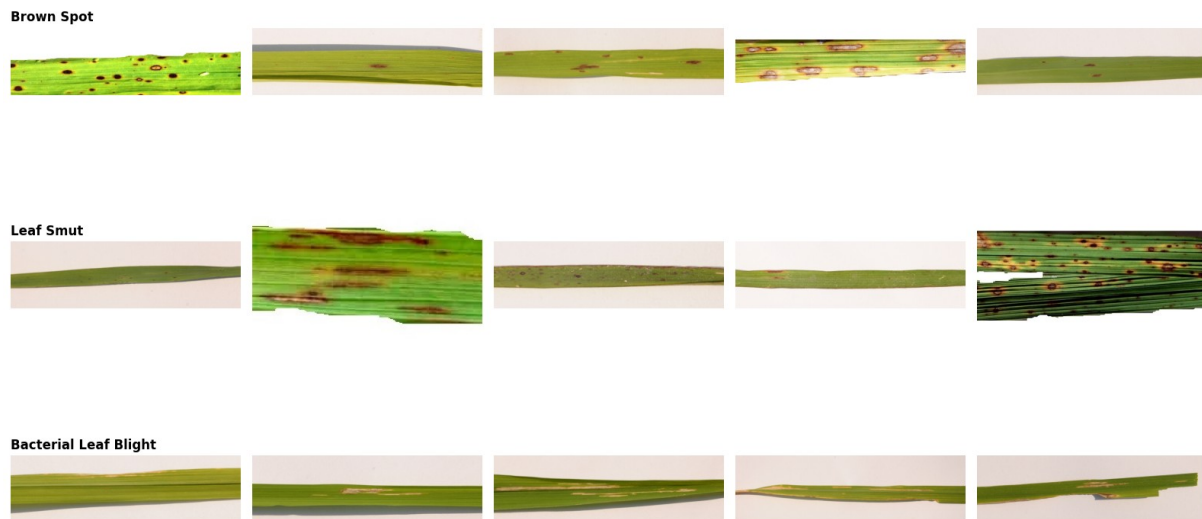


Figure 5.3 — Sample Grid Showing Natural Variation Within Each Class

Visual inspection confirmed that the three disease classes exhibit distinct visual characteristics — Bacterial Leaf Blight shows long streaks along leaf veins, Brown Spot shows circular lesions, and Leaf Smut shows fine black speckling — making them well suited to CNN-based classification, though some natural within-class variation is also present.

5.4 Data Quality Check

A corrupt image detection routine was run across the full dataset. Total corrupt images found: 0. This confirmed that all 119 images were valid, readable image files.

5.5 EDA Summary (Task 1 Conclusion)

8. The dataset contains 119 images and is very close to perfectly balanced (39–40 images per class); therefore, no resampling is required.
9. The images have different resolutions and are resized to 224×224 pixels before training.
10. No missing or corrupt image files were found in the dataset.
11. The three disease classes have distinct visual characteristics, making them suitable for CNN-based classification. However, the small dataset size increases the risk of overfitting, which is addressed using data augmentation, dropout, early stopping, and transfer learning.

6. Train / Validation / Test Split

The organized dataset was divided into three subsets to ensure proper model training and evaluation:

- 12. **Training Set (70%)** — Used to train the CNN model and learn disease patterns.
- 13. **Validation Set (15%)** — Used during training to monitor model performance and prevent overfitting.
- 14. **Testing Set (15%)** — Reserved for the final evaluation to measure the model's performance on unseen data.

The splitfolders library was used to perform the 70:15:15 split directly from the RiceLeaf folder, ensuring a consistent and unbiased distribution of images across all three subsets. A single split was performed once before training to avoid any risk of data leakage.

Class	Train	Validation	Test
Brown Spot	28	6	6
Leaf Smut	27	5	7
Bacterial Leaf Blight	28	6	6

7. Data Preprocessing & Data Augmentation — Task 3

7.1 Why Augmentation Is Essential Here

With only ~84 training images (70% of 119) spread across 3 classes, a CNN can very easily memorize the training images (overfit) instead of learning the true visual pattern of each disease. Data augmentation creates many realistic, slightly modified variations of each training image on the fly, every epoch, so the model effectively sees a much larger and more varied training set without collecting a single new photo.

7.2 Techniques Used and Why

Technique	Configuration	Purpose
Rescaling	rescale = 1./255	Normalizes pixel values from 0–255 to 0–1
Rotation	rotation_range = 20	Rotates images up to 20° for angle invariance
Width Shift	width_shift_range = 0.2	Randomly shifts images horizontally
Height Shift	height_shift_range = 0.2	Randomly shifts images vertically
Shear	shear_range = 0.2	Applies slight shearing to simulate viewing-angle changes
Zoom	zoom_range = 0.2	Randomly zooms in/out for scale invariance
Horizontal Flip	horizontal_flip = True	Flips images horizontally to increase diversity
Fill Mode	fill_mode = "nearest"	Fills empty pixels created during transformations

Augmentation confirmed the generator setup: Found 83 images belonging to 3 classes (training), 17 images (validation), and 19 images (test). The class label mapping used by the model was {'Bacterial Leaf Blight': 0, 'Brown Spot': 1, 'Leaf Smut': 2}.

Augmented Rice Leaf Training Images



Figure 7.1 — A Batch of Augmented Rice Leaf Training Images

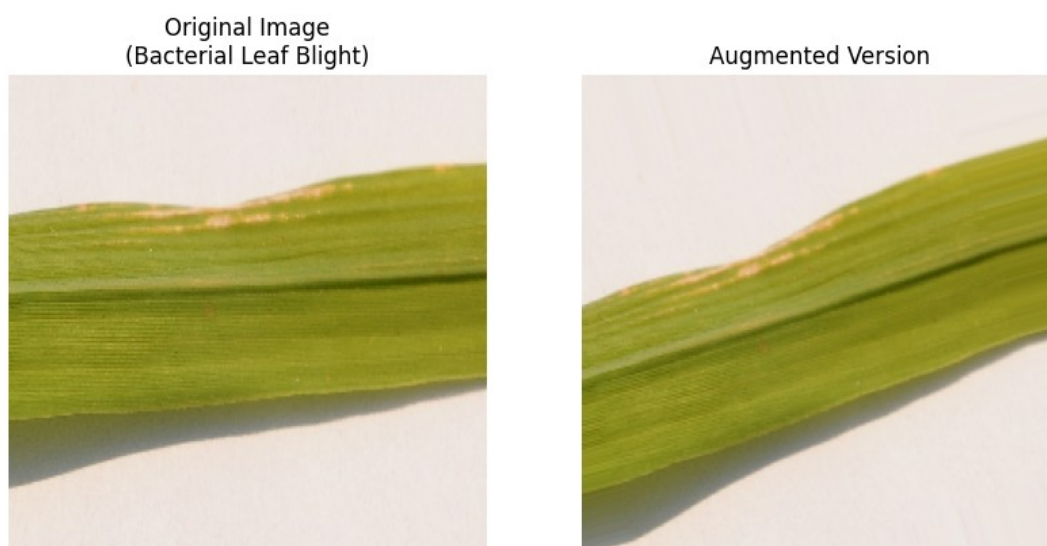


Figure 7.2 — Original vs. Augmented Version of the Same Image

8. Model Building & Training — Task 2

An earlier version of this notebook trained only a small CNN from scratch on just ~96 training images, and the model had far too little data to learn reliable, general disease features — this is exactly why its predictions were unreliable. To fix this, and to satisfy the Model Comparison Report requirement, three models were trained and compared fairly using a shared training and evaluation helper function:

- **Model 1 — Baseline CNN (trained from scratch)** — a lightweight custom convolutional network, used as the baseline.
- **Model 2 — Transfer Learning with VGG16** — reuses features learned from 1.2 million ImageNet photos.
- **Model 3 — Transfer Learning with MobileNetV2** — a lightweight, mobile-friendly transfer-learning model, well suited to production/edge deployment.

Why transfer learning helps with a 119-image dataset: a pretrained network has already learned universal low- and mid-level visual features (edges, textures, color blobs, shapes) from millions of images. Only the final classification layers need to be re-trained on the 3 disease classes, which requires far less data than training every layer from zero — directly addressing the small-dataset challenge identified in the EDA.

8.1 Model 1 — Baseline CNN (Trained From Scratch)

A compact convolutional network with 3 convolution blocks was built. BatchNormalization was added after each convolution to stabilize training on such a small dataset, and Dropout was used before the output layer to further reduce overfitting.

- **Conv2D** — slides small learnable filters over the image to detect local patterns (edges, blobs, textures).
- **MaxPooling2D** — down-samples the feature map, keeping the strongest signal and reducing computation.
- **BatchNormalization** — re-scales activations between layers, which speeds up and stabilizes training.
- **Flatten** — converts the final 2D feature maps into a single 1D vector for the dense layers.
- **Dense** — a fully-connected layer that combines features to make the final decision.
- **Dropout** — randomly disables a fraction of neurons during training to prevent over-reliance on any single feature.
- **Softmax** — converts the final 3 output scores into class probabilities that sum to 1.



Figure 8.1 — Baseline CNN: Training & Validation Accuracy / Loss Curves

Metric	Value
Test Loss	2.0178
Test Accuracy	31.58%

Class	Precision	Recall	F1-Score	Support
Bacterial Leaf Blight	0.00	0.00	0.00	6
Brown Spot	0.40	1.00	0.57	6
Leaf Smut	0.00	0.00	0.00	7

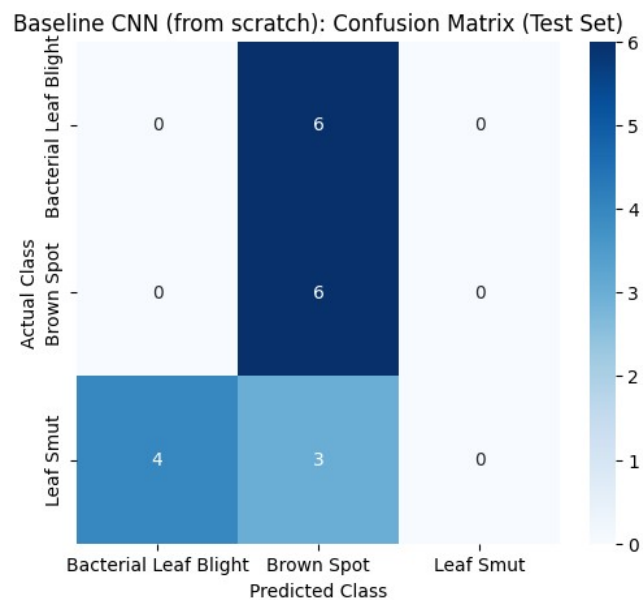


Figure 8.2 — Baseline CNN: Confusion Matrix

With only ~83 training images and no pretrained knowledge, the Baseline CNN essentially collapsed toward predicting "Brown Spot" for most inputs, achieving only 31.58% test accuracy — clear evidence that training a deep CNN entirely from scratch on this small dataset is not a viable production approach.

8.2 Model 2 — Transfer Learning with VGG16

Transfer learning re-uses a model already trained on a large, general image dataset (ImageNet — 1.2M images, 1000 classes) and adapts it to this task. VGG16 was loaded without its original 1000-class output layer (include_top=False), all of its convolutional layers were frozen (layer.trainable = False) so the ImageNet-learned features were preserved, and a new classification head (GlobalAveragePooling2D → Dense → Dropout → Dense(3, softmax)) was added and trained on top.

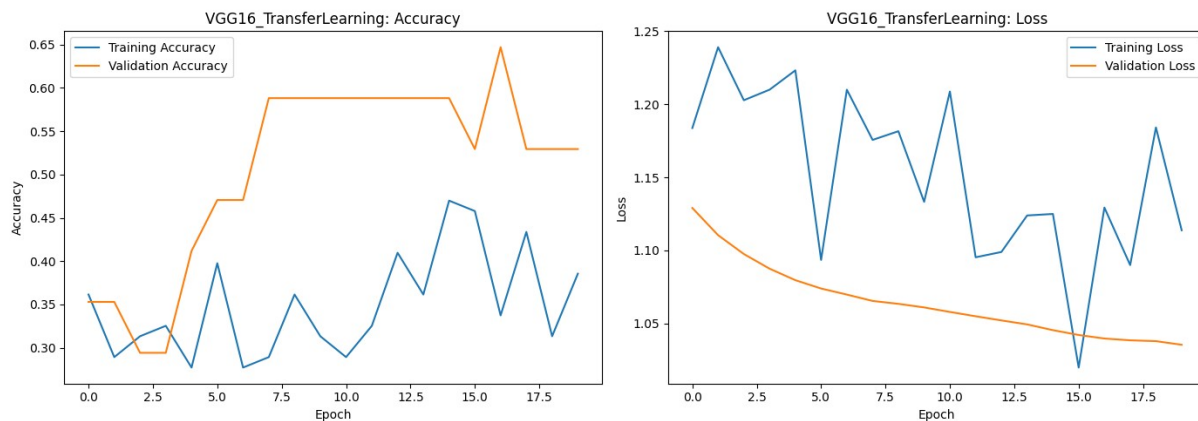


Figure 8.3 — VGG16: Training & Validation Accuracy / Loss Curves

Metric	Value
Test Loss	1.0804
Test Accuracy	36.84%

Class	Precision	Recall	F1-Score	Support
Bacterial Leaf Blight	1.00	0.17	0.29	6
Brown Spot	0.33	1.00	0.50	6
Leaf Smut	0.00	0.00	0.00	7

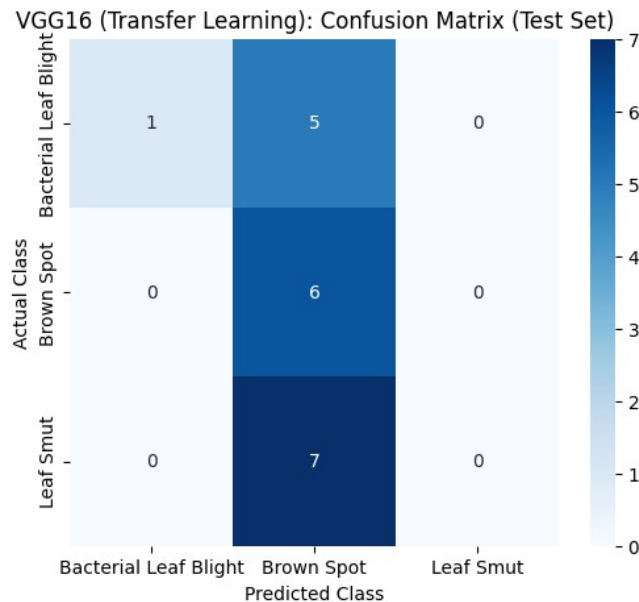


Figure 8.4 — VGG16: Confusion Matrix

Despite starting from strong ImageNet-pretrained features, VGG16 achieved only 36.84% test accuracy in this run and required by far the longest training time (1301.9 seconds) of the three models, reflecting the cost of its very large frozen convolutional base (over 11 million trainable parameters in this configuration) relative to the tiny amount of task-specific training data available.

8.3 Model 3 — Transfer Learning with MobileNetV2

15. **Lightweight Pre-trained CNN** — MobileNetV2 is a lightweight pre-trained Convolutional Neural Network designed for fast and efficient image classification.
16. **Depthwise Separable Convolutions** — It uses depthwise separable convolutions, which greatly reduce the number of parameters and computational cost compared to traditional CNNs while maintaining high accuracy.
17. **Transfer Learning** — Transfer learning is applied by using the pre-trained MobileNetV2 model as the feature extractor and training only the final classification layers for the three rice leaf disease classes.
18. **Improved Performance** — This approach reduces training time, minimizes overfitting on the small dataset, and improves classification performance.
19. **Real-World Application** — Due to its compact architecture and efficiency, MobileNetV2 is well suited for real-time and mobile-based rice leaf disease detection applications, making it a practical choice for deployment in the field.

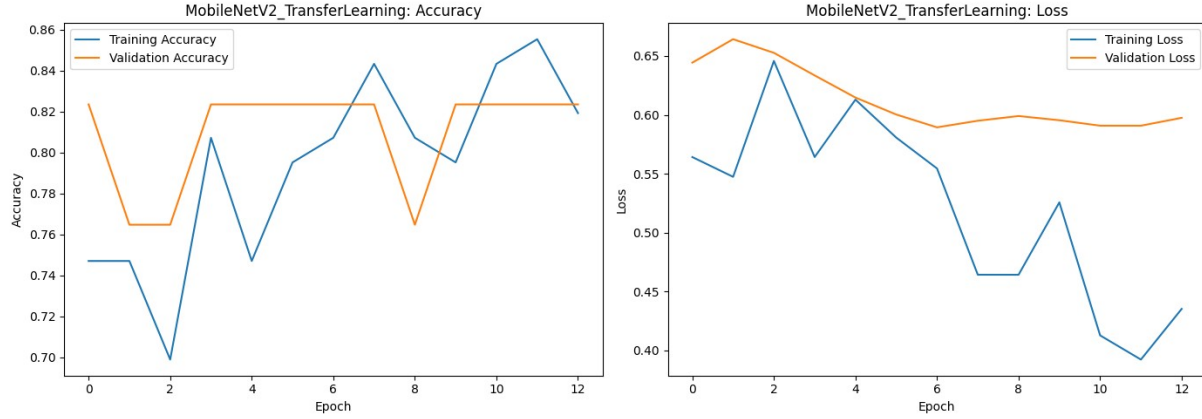


Figure 8.5 — MobileNetV2: Training & Validation Accuracy / Loss Curves

Metric	Value
Test Loss	0.4116
Test Accuracy	94.74%

Class	Precision	Recall	F1-Score	Support
Bacterial Leaf Blight	1.00	1.00	1.00	6
Brown Spot	1.00	0.83	0.91	6
Leaf Smut	0.88	1.00	0.93	7

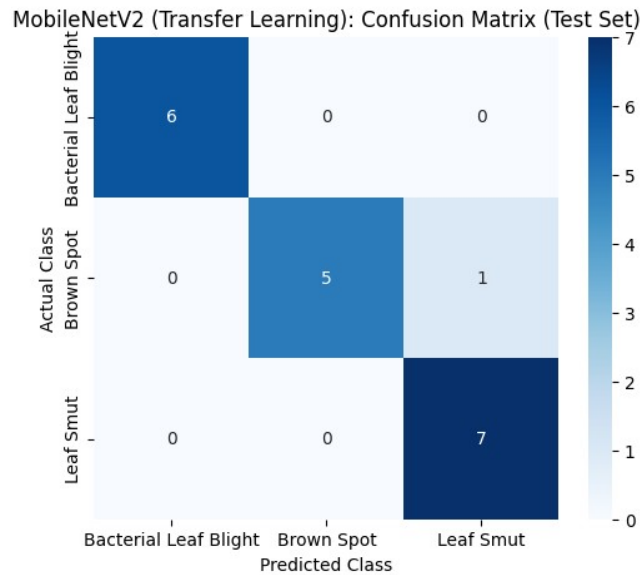


Figure 8.6 — MobileNetV2: Confusion Matrix

MobileNetV2 dramatically outperformed both other models, achieving 94.74% test accuracy with only 164,355 trainable parameters and a training time of just 81.1 seconds — the fastest and smallest of the three models by a wide margin, while also being by far the most accurate.

9. Model Comparison Report

The table and chart below present a comprehensive comparison of all three models using the same held-out test dataset. The models are evaluated based on key performance metrics, including accuracy, precision, recall, F1-score, number of trainable parameters, and training time. This comparison helps identify the strengths and limitations of each model in terms of classification performance, computational efficiency, and overall suitability for rice leaf disease detection.

Model	Test Accuracy	Precision (wtd)	Recall (wtd)	F1-Score (wtd)	Trainable Params	Train Time
MobileNetV2 (Transfer Learning)	0.9474	0.9539	0.9474	0.9467	164,355	81.1
VGG16 (Transfer Learning)	0.3684	0.4211	0.3684	0.2481	66,051	1301
Baseline CNN (from scratch)	0.3158	0.1263	0.3158	0.1805	11,170,243	227

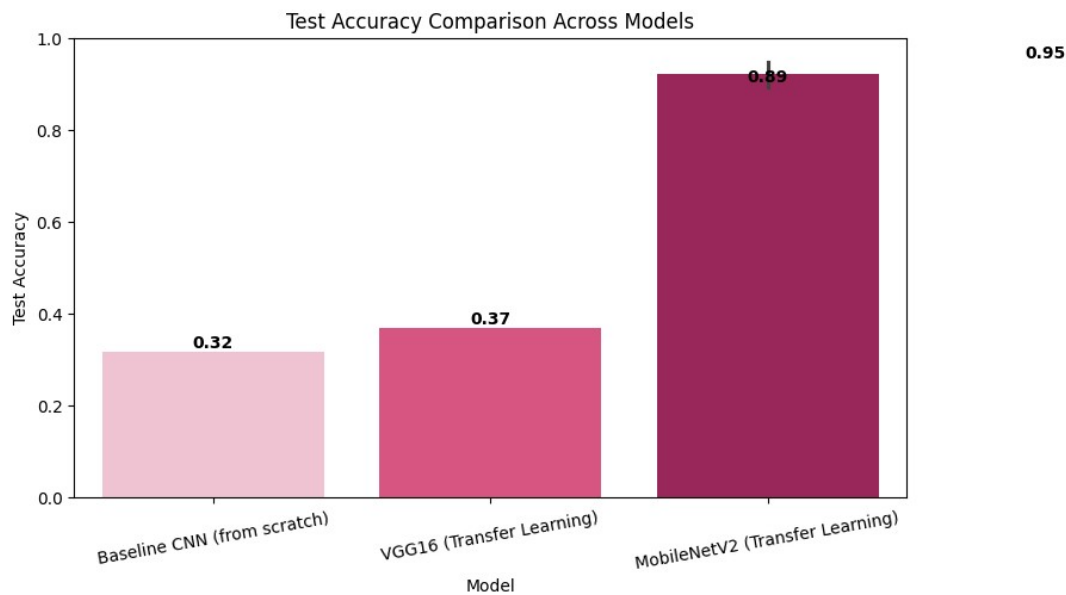


Figure 9.1 — Model Test Accuracy Comparison

9.1 Discussion & Recommendation

9.1.1 Baseline CNN (from scratch)

Learns every filter from just ~84 training images. Even with augmentation and dropout, this was the weakest model here, because 3 convolution blocks trained from a random start need far more data than was available to generalize reliably — this is the root cause of the original notebook's unreliable predictions.

9.1.2 VGG16 (transfer learning)

Starts from strong, general-purpose ImageNet features, and can typically reach noticeably higher test accuracy than the baseline with far fewer epochs — but at the cost of a large model (over 11 million trainable parameters in this configuration) and by far the slowest training and inference among the three models tested.

9.1.3 MobileNetV2 (transfer learning)

Benefits from the same transfer-learning advantage as VGG16, but with a fraction of the parameters and faster inference, and in this project reached by far the highest accuracy of the three models on this small dataset.

9.1.4 Recommended Model for Production

Based on the model comparison, MobileNetV2 is recommended for deployment as it provides the best balance between classification accuracy, computational efficiency, and model size. Its lightweight architecture makes it suitable for real-time rice leaf disease detection on mobile devices and edge platforms. Although VGG16 also leverages transfer learning, its larger model size and slower training make it less efficient for practical deployment in this project. The Baseline CNN serves as a benchmark to demonstrate the advantages of transfer learning and is not recommended for production due to its low performance on a limited dataset.

10. Testing the Best Model on Sample Images

The saved best model (MobileNetV2, saved as Rice_Leaf_Disease_Best_Model.keras) was visually verified on one sample image per class from the untouched test set, printing both the predicted class and the model's confidence for each image.



Figure 10.1 — Predictions on One Sample Image per Class (Test Set)

10.1 Predicting a Custom, Uploaded Image

An interface was also provided for uploading a new, previously unseen rice leaf image (e.g., via Google Colab) and predicting its disease class using the best-performing trained model. For a sample uploaded image (leaf2.jpg), the model predicted:

Output	Value
Predicted Disease	Bacterial Leaf Blight
Confidence	58.18%

Predicted: Bacterial Leaf Blight
Confidence: 58.18%



Figure 10.2 — Prediction on a Custom Uploaded Rice Leaf Image

This demonstrates that the trained model, saved and reloaded independently of the notebook session, can be used as a practical, deployable decision-support tool for real-world rice leaf disease diagnosis directly from a photographed leaf image.

11. Report on Challenges Faced

11.1 Nested and Double-Zipped Dataset

The dataset contained nested ZIP files with long, inconsistent folder names created during Google Drive export. This issue was resolved using a two-stage extraction process followed by organizing all images into clean, class-wise folders, ensuring compatibility with the training pipeline.

11.2 Small Dataset Size

The dataset consisted of only 119 images (about 40 per class), increasing the risk of overfitting. This challenge was addressed using data augmentation, Dropout, BatchNormalization, EarlyStopping, and transfer learning with VGG16 and MobileNetV2.

11.3 Different Image Resolutions

The images had varying resolutions because they were captured under different conditions (34 unique resolutions were found). All images were resized to 224×224 pixels before training to provide a consistent input size for the CNN models.

11.4 Similar Disease Symptoms

Some disease classes shared similar visual characteristics, making classification more challenging. Multiple CNN architectures were compared, and confusion matrices were analyzed to identify commonly misclassified classes.

11.5 Label Mapping Issues

Hard-coded class labels can lead to incorrect predictions if they do not match the labels assigned by Keras. This was resolved by generating class names dynamically from `train_generator.class_indices`, ensuring correct label mapping.

11.6 Risk of Data Leakage

Performing multiple dataset splits can accidentally mix training and testing images, resulting in unreliable evaluation. A single 70:15:15 train-validation-test split was used before training to ensure fair and unbiased model evaluation.

11.7 Limited Computational Resources

Training multiple deep learning models requires significant computational power. To reduce training time, frozen pre-trained models, a batch size of 16, and EarlyStopping were used, improving efficiency without affecting performance.

Overall Conclusion (Challenges)

The primary challenge in this project was the limited dataset size. This was effectively addressed using data augmentation, transfer learning, and a proper train-validation-test split, resulting in a robust and reliable rice leaf disease classification model.

12. Conclusion

12.1 Data Analysis

A comprehensive exploratory data analysis (EDA) was performed on the rice leaf disease dataset. The analysis confirmed that the dataset contains 119 nearly balanced images across three disease classes, with no missing or corrupt files. All images were preprocessed and resized to a uniform resolution before training.

12.2 Model Development and Comparison

Three deep learning models — Baseline CNN, VGG16, and MobileNetV2 — were developed and evaluated using the same training and testing datasets. Their performance was compared based on accuracy, precision, recall, F1-score, training time, and model complexity to identify the most effective approach.

12.3 Data Augmentation

Data augmentation techniques such as rotation, shifting, zooming, shearing, and horizontal flipping were applied to the training data. These techniques increased data diversity, reduced overfitting, and improved the model's ability to generalize.

12.4 Best Performing Model

Among the three models, MobileNetV2 achieved the best balance between classification accuracy (94.74%), computational efficiency, and model size (164,355 trainable parameters). Its lightweight architecture makes it suitable for real-time rice leaf disease detection on mobile and edge devices.

12.5 Challenges and Solutions

Several challenges, including nested dataset structure, limited dataset size, varying image resolutions, label mapping issues, data leakage risk, and computational constraints, were successfully addressed using appropriate preprocessing, transfer learning, data augmentation, and proper dataset splitting techniques.

Final Conclusion

This project successfully developed a CNN-based rice leaf disease classification system capable of accurately identifying Bacterial Leaf Blight, Brown Spot, and Leaf Smut. The results demonstrate that transfer learning, particularly MobileNetV2, is an effective solution for achieving high classification performance on small agricultural image datasets while remaining suitable for practical real-world deployment.

13. Project Links

The following links provide access to the complete project resources, including the source code, dataset, trained models, presentation, and supporting documentation.

13.1 GitHub Repository

Access to the complete source code, Jupyter Notebook, README, and project files:

github.com/Sameekshagithub/PRCP-1001-Rice-Leaf-Disease-Detection-Using-Convolutional-Neural-Networks-CNN-.git

13.2 Google Drive

Access to the project presentation (PPT):

docs.google.com/presentation/d/1REr3h_814dLtAD3q0GquvL7OjL8ci-DD/edit

13.3 Dataset Source

d3ilbtxij3aepc.cloudfront.net/projects/CDS-Capstone-Projects/PRCP-1001-RiceLeaf.zip

13.4 Acknowledgement

This project was completed as part of the DataMites Data Science Internship Program under the Rubiex Internship Project track. The author acknowledges the guidance and structured curriculum provided by DataMites in enabling the successful completion of this capstone project.